

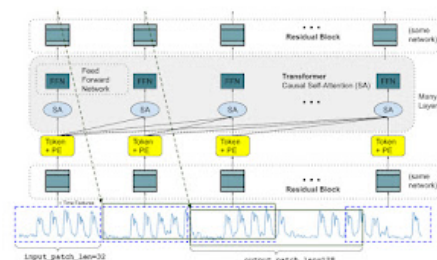


[Главная](#) > [Блог](#) >

Базовая модель для прогнозирования временных рядов с использованием только декодера

2 февраля 2024 года

Опубликовано Раджатом Сенем и Ичэнем Чжоу, Google Research



ОБНОВЛЕНИЕ • 8 МАЯ 2024 Г.

Статья [«Базовая модель только для декодирования при прогнозировании временных рядов»](#) была принята к публикации на [ICML 2024](#), а графики были обновлены для визуализации последних результатов. Модель теперь доступна в наших репозиториях [HuggingFace](#) и [GitHub](#).

TimesFM — это модель прогнозирования, предварительно обученная на большом массиве временных рядов, состоящем из 100 миллиардов реальных временных точек. Она демонстрирует впечатляющие результаты на различных общедоступных тестах из разных областей и с разной степенью детализации.

[БЫСТРЫЙ ПЕРЕХОД](#)

Прогнозирование временных рядов широко применяется в различных областях, таких как розничная торговля, финансы, производство, здравоохранение и естественные науки. Например, в сфере розничной торговли было замечено, что повышение точности прогнозирования спроса может существенно снизить затраты на хранение товаров и увеличить прибыль. Модели глубокого обучения (deep learning, DL) стали популярным подходом к прогнозированию сложных многомерных данных временных рядов, поскольку они хорошо зарекомендовали себя в различных условиях (например, модели глубокого обучения показали хорошие результаты в конкурсе M5).

В то же время наблюдается стремительный прогресс в области больших языковых моделей, используемых для решения задач обработки естественного языка (natural language processing, NLP), таких как перевод, генерация с использованием результатов поиска и автодополнение кода. Эти модели обучаются на огромных массивах текстовых данных, полученных из различных источников, таких как общедоступные данные и открытый исходный код, что позволяет им выявлять закономерности в языках. Это делает их очень мощными инструментами нулевого обучения; например, в сочетании с поиском они могут отвечать на вопросы о текущих событиях и обобщать информацию о них.

Несмотря на то, что системы прогнозирования на основе глубокого обучения в значительной степени превосходят традиционные методы, а также на прогресс в снижении затрат на обучение и логический вывод, они сталкиваются с трудностями: большинство архитектур глубокого обучения требуют долгих и сложных циклов обучения и проверки, прежде чем заказчик сможет протестировать модель на новых временных рядах. Базовая модель для прогнозирования временных рядов, напротив, может выдавать приемлемые прогнозы на основе ранее не встречавшихся данных без дополнительного обучения, что позволяет пользователям

To that end, in [“A decoder-only foundation model for time-series forecasting”](#), accepted at [ICML 2024](#), we introduce TimesFM, a single forecasting model pre-trained on a large time-series corpus of 100 billion real world time-points. Compared to the latest large language models (LLMs), TimesFM is much smaller (200M parameters), yet we show that even at such scales, its zero-shot performance on a variety of unseen datasets of different domains and temporal granularities come close to the state-of-the-art supervised approaches trained explicitly on these datasets. To access the model, please visit our [HuggingFace](#) and [GitHub](#) repos.

A decoder-only foundation model for time-series forecasting

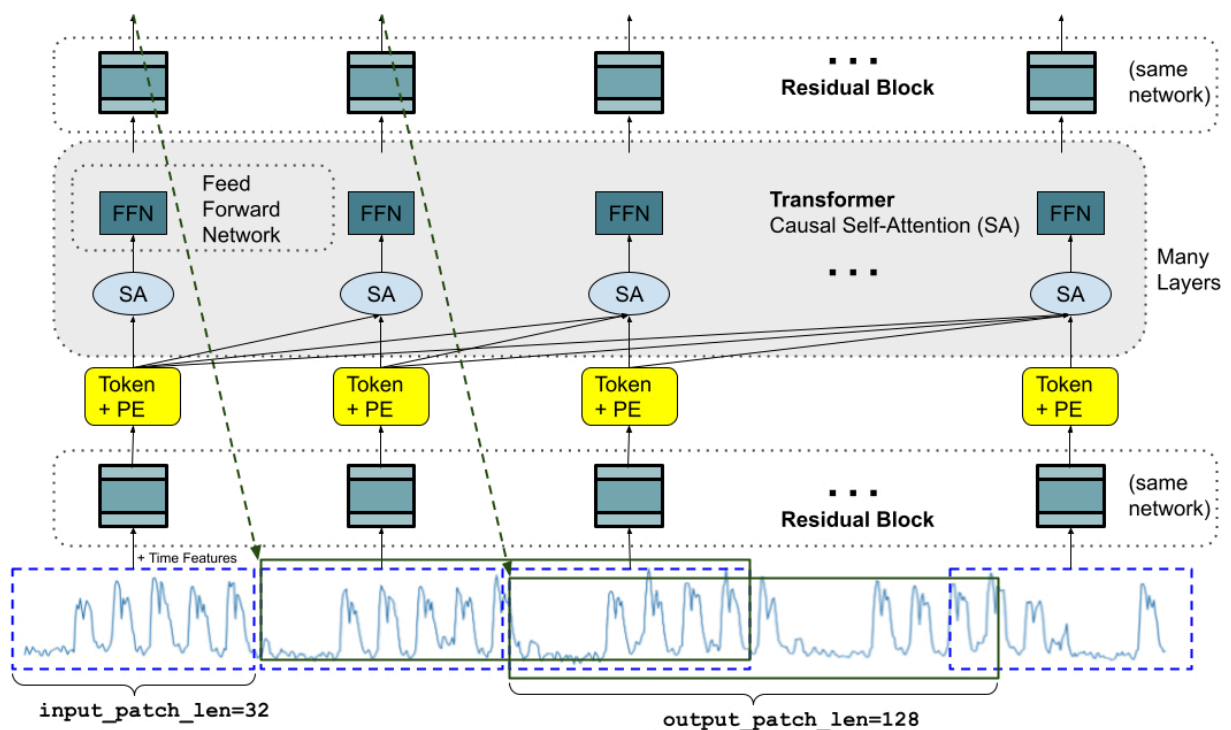
LLMs are usually trained in a [decoder-only](#) fashion that involves three steps. First, text is broken down into subwords called tokens. Then, the tokens are fed into stacked causal [transformer](#) layers that produce an output corresponding to each input token (it cannot attend to future tokens). Finally, the output corresponding to the i -th token summarizes all the information from previous tokens and predicts the $(i+1)$ -th token. During inference, the LLM generates the output one token at a time. For example, when prompted with “What is the capital of France?”, it might generate the token “The”, then condition on “What is the capital of France? The” to generate the next token “capital” and so on until it generates the complete answer: “The capital of France is Paris”.

A foundation model for time-series forecasting should adapt to variable context (what we observe) and horizon (what we query the model to forecast) lengths, while having enough capacity to encode all patterns from a large pretraining dataset. Similar to LLMs, we use stacked transformer layers (self-attention and [feedforward](#) layers) as the main building blocks for the TimesFM model. In the context of time-series forecasting, we treat a patch (a group of contiguous time-points) as a token that was popularized by a recent [long-horizon forecasting work](#). The task then is to forecast the $(i+1)$ -th patch of time-points given the i -th output at the end of the stacked transformer layers.

However, there are several key differences from language models. Firstly, we need a [multilayer perceptron](#) block with residual connections to convert a patch of time-series into a token that can be input to the transformer layers along with [positional encodings](#) (PE). For that, we use a residual block similar to our prior work in [long-horizon forecasting](#). Secondly, at the other end, an output token

Исследования

Consider a time-series of length 512 time-points being used to train a TimesFM model with input patch length 32 and output patch length 128. During training, the model is simultaneously trained to use the first 32 time-points to forecast the next 128 time-points, the first 64 time-points to forecast time-points 65 to 192, the first 96 time-points to forecast time-points 97 to 224 and so on. During inference, suppose the model is given a new time-series of length 256 and tasked with forecasting the next 256 time-points into the future. The model will first generate the future predictions for time-points 257 to 384, then condition on the initial 256 length input plus the generated output to generate time-points 385 to 512. On the other hand, if in our model the output patch length was equal to the input patch length of 32 then for the same task we would have to go through eight generation steps instead of just the two above. This increases the chances of more errors accumulating and therefore, in practice, we see that a longer output patch length yields better performance for long-horizon forecasting



TimesFM architecture.

Pretraining data

Исследования

we have found works best:

Synthetic data helps with the basics. Meaningful synthetic time-series data can be generated using statistical models or physical simulations. These basic temporal patterns can teach the model the grammar of time series forecasting.

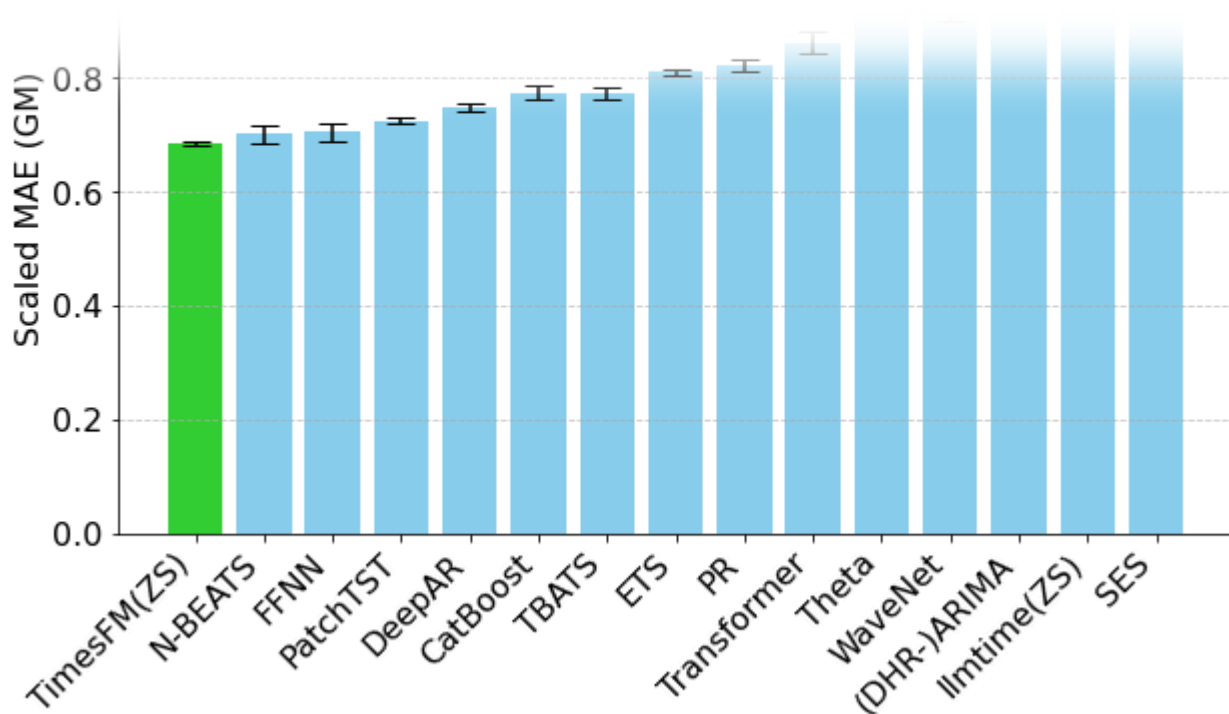
Real-world data adds real-world flavor. We comb through available public time series datasets, and selectively put together a large corpus of 100 billion time-points. Among these datasets there are [Google Trends](#) and [Wikipedia Pageviews](#), which track what people are interested in, and that nicely mirrors trends and patterns in many other real-world time series. This helps TimesFM understand the bigger picture and generalize better when provided with domain-specific contexts not seen during training.

Zero-shot evaluation results

We evaluate TimesFM zero-shot on data not seen during training using popular time-series benchmarks. We observe that TimesFM performs better than most statistical methods like [ARIMA](#), [ETS](#) and can match or outperform powerful DL models like [DeepAR](#), [PatchTST](#) that have been *explicitly trained* on the target time-series.

We used the [Monash Forecasting Archive](#) to evaluate TimesFM's out-of-the-box performance. This archive contains tens of thousands of time-series from various domains like traffic, weather, and demand forecasting covering frequencies ranging from few minutes to yearly data. Following existing literature, we inspect the [mean absolute error](#) (MAE) [appropriately scaled](#) so that it can be averaged across the datasets. We see that zero-shot (ZS) TimesFM is better than most supervised approaches, including recent deep learning models. We also compare TimesFM to [GPT-3.5](#) for forecasting using a specific prompting technique proposed by [llmtime\(ZS\)](#). We demonstrate that TimesFM performs better than llmtime(ZS) despite being orders of magnitude smaller.

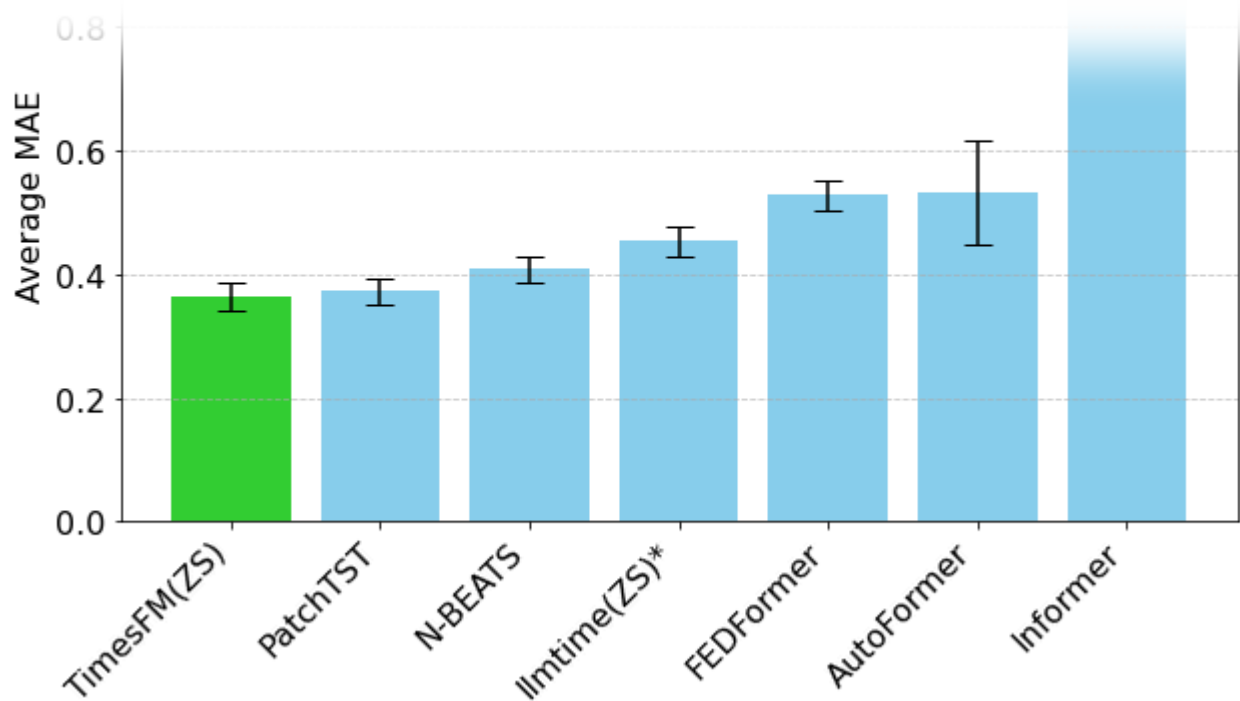
Исследования



Geometric mean (GM, and why we do so) of Scaled MAE (the lower the better) of TimesFM(ZS) against other supervised and zero-shot approaches on Monash datasets.

Most of the Monash datasets are short or medium horizon, i.e., the prediction length is not too long. We also test TimesFM on popular benchmarks for long horizon forecasting against a recent state-of-the-art baseline PatchTST (and other long-horizon forecasting baselines). In the next figure, we plot the MAE on ETT datasets for the task of predicting 96 and 192 time-points into the future. The metric has been calculated on the last test window of each dataset (as done by the Ilmtime paper). We see that TimesFM not only surpasses the performance of Ilmtime(ZS) but also matches that of the supervised PatchTST model explicitly trained on the respective datasets.

Исследования



Last window MAE (the lower the better) of TimesFM(ZS) against Ilmtime(ZS) and long-horizon forecasting baselines on ETT datasets.

Conclusion

We train a decoder-only foundation model for time-series forecasting using a large pretraining corpus of 100B real world time-points, the majority of which was search interest time-series data derived from Google Trends and pageviews from Wikipedia. We show that even a relatively small 200M parameter pretrained model that uses our TimesFM architecture displays impressive zero-shot performance on a variety of public benchmarks from different domains and granularities.

Acknowledgements

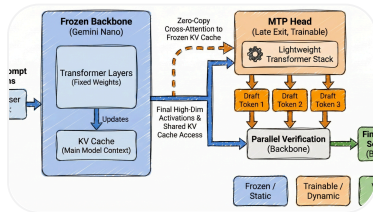
This work is the result of a collaboration between several individuals across Google Research and Google Cloud, including (in alphabetical order):

Abhimanyu Das, Weihao Kong, Andrew Leach, Mike Lawrence, Alex Martin, Rajat Sen, Yang Yang, Skander Hannachi, Ivan Kuznetsov and Yichen Zhou.

Labels:

[Machine Intelligence](#)

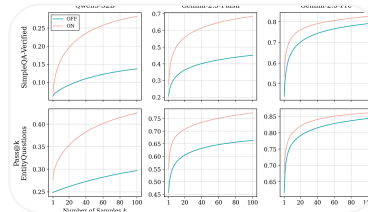
Other posts of interest



JUNE 26, 2026

Accelerating Gemini Nano models on Pixel with frozen Multi-Token Prediction

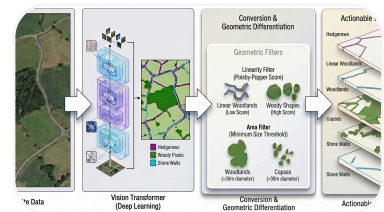
Machine Intelligence ·
Mobile Systems ·
Natural Language Processing



JUNE 24, 2026

Thinking to recall: How reasoning unlocks parametric knowledge in LLMs

Generative AI ·
Machine Intelligence ·
Natural Language Processing



JUNE 16, 2026

From pixels to planning: Earth AI for nature restoration

Climate & Sustainability ·
Earth AI ·
Machine Intelligence ·
Open Source Models & Datasets

Follow us



Explore our other initiatives

Google AI

Discover how Google AI is committed to enriching knowledge and solving complex challenges

Products

Build

Research

Responsibility

Societal Impact

About

Google Cloud

High-performance infrastructure for cloud computing, data analytics & machine learning

Overview

Solutions

Products

Pricing

Resources

Исследования

humanity

Labs

Models

About

Research

Experiments

Science

Stay connected

About

Google

About Google

Google Products

Privacy

Terms